

1 Introduction to Computer Graphics

Computer Graphics is the field of computer science that focuses on generating, manipulating, and displaying visual content using computers. It encompasses creating images, animations, and interactive visualizations for applications ranging from video games to scientific simulations. Computer graphics combines mathematics, algorithms, and hardware to render 2D and 3D scenes efficiently.

1.1 Key Concepts

- **Goals:** Produce realistic or stylized visuals, enable real-time interaction, and optimize for performance.
- **Types:**
 - **2D Graphics:** Images, sprites, UI (e.g., Photoshop, web design).
 - **3D Graphics:** Models, scenes (e.g., games, CAD).
 - **Real-Time:** Fast rendering for games (60 FPS).
 - **Offline:** High-quality rendering for movies (e.g., Pixar).
- **Applications:** Gaming (Unreal Engine), film (CGI), VR/AR, medical visualization, architecture, and data visualization.
- **As of 2025:** Trends include real-time ray tracing, AI-driven rendering (e.g., neural rendering, DLSS 3), integration with generative AI for asset creation, and quantum graphics algorithms for specialized tasks.

1.2 History

- **1960s:** Ivan Sutherland's Sketchpad (1963), first interactive graphics system.
- **1970s–1980s:** 3D graphics emerge (Utah teapot, 1975). Framebuffers and GPUs (NVIDIA, 1999).
- **1990s:** OpenGL (1992), DirectX (1995). Toy Story (1995) showcases CGI.
- **2000s:** Programmable shaders (GPU programming). Real-time 3D in games (Half-Life 2).
- **2010s:** Physically-based rendering (PBR), VR (Oculus Rift, 2016).
- **2020s:** Ray tracing (NVIDIA RTX, 2018), AI upscaling, neural graphics (e.g., Neural Radiance Fields, NeRF). By 2025, sustainable rendering and quantum-assisted graphics are emerging.

2 Foundations of Computer Graphics

2.1 Mathematical Foundations

- **Linear Algebra:** Vectors (position, direction), matrices (transformations: rotation, scaling, translation).
- **Geometry:** Points, lines, polygons, curves (Bézier, NURBS).
- **Calculus:** For shading, animation (e.g., motion curves).
- **Probability:** For Monte Carlo methods in rendering (e.g., path tracing).

2.2 Graphics Pipeline

The process to render a 3D scene to a 2D screen:

1. **Modeling:** Define 3D objects (vertices, meshes).
2. **Transformation:** Position objects (world, view, projection).
3. **Rasterization:** Convert 3D to 2D pixels.
4. **Shading:** Compute lighting, textures.
5. **Output:** Display on screen.

2.3 Hardware

- **GPU (Graphics Processing Unit):** Parallel processing (e.g., NVIDIA A100, AMD Radeon). CUDA/OpenCL for general-purpose computing.
- **TPU/Neuromorphic:** Emerging for AI-driven graphics.
- **Display:** Monitors, VR headsets (e.g., 4K OLED, 120Hz).

3 2D Graphics

3.1 Basics

- **Raster Graphics:** Pixel-based (e.g., PNG, JPEG). Tools: Photoshop, GIMP.
- **Vector Graphics:** Mathematical shapes (e.g., SVG). Tools: Illustrator, Inkscape.
- **Primitives:** Lines, circles, polygons.

3.2 Techniques

- **Line Drawing:** Bresenham's algorithm (efficient integer-based).
- **Clipping:** Cohen-Sutherland, Liang-Barsky for viewport.
- **Transformations:** Translation, scaling, rotation via 2x2 matrices.
- **Anti-Aliasing:** Smooth edges (e.g., supersampling).

Example (Drawing a Line in Python with Pygame):

```
import pygame
pygame.init()
screen = pygame.display.set_mode((800, 600))
screen.fill((255, 255, 255)) # White background
pygame.draw.line(screen, (0, 0, 255), (100, 100), (500, 500), 2) #
    Blue line
pygame.display.flip()
pygame.time.wait(2000)
pygame.quit()
```

4 3D Graphics

4.1 Modeling

- **Vertices/Edges/Faces:** Meshes define shapes (e.g., triangles).
- **Formats:** OBJ, FBX, STL.
- **Tools:** Blender, Maya, 3ds Max.
- **Curves/Surfaces:** Bézier, B-Splines, NURBS for smooth surfaces.

4.2 Transformations

- **Model Transform:** Object to world space.
- **View Transform:** Camera perspective (look-at matrix).
- **Projection:** Orthographic (parallel) or perspective (3D to 2D).
- **Matrices:** 4x4 homogeneous for 3D (e.g., translate $[x,y,z] \rightarrow [x,y,z,1]$).

4.3 Coordinate Systems

- **Object Space:** Local to model.
- **World Space:** Global scene.
- **View Space:** Camera-centered.
- **Clip Space:** After projection, before viewport.

5 Rendering

Converting 3D models to 2D images.

5.1 Rasterization

- **Process:** Project 3D triangles to 2D, fill pixels.
- **Pipeline** (OpenGL/DirectX):
 - Vertex Shader: Transform vertices.
 - Tessellation: Subdivide geometry (optional).
 - Geometry Shader: Modify primitives.
 - Rasterizer: Convert to fragments (pixels).
 - Fragment Shader: Compute color, texture.
- **Z-Buffering:** Depth test to handle occlusion.

5.2 Ray Tracing

- **Process:** Trace rays from camera through pixels, compute intersections, lighting.
- **Types:**
 - Ray Casting: Basic visibility.
 - Path Tracing: Monte Carlo for global illumination (shadows, reflections).
- **Pros:** Photorealistic (reflections, refractions).
- **Cons:** Compute-intensive; real-time with RTX GPUs (2025).
- **Example:** NVIDIA OptiX, Unreal Engine 5's Lumen.

5.3 Shading and Lighting

- **Local Models:**
 - **Lambertian:** Diffuse (matte surfaces).
 - **Phong:** Diffuse + specular highlights.
- **Global Illumination:** Ray tracing, radiosity for realistic light (e.g., caustics).
- **Physically-Based Rendering (PBR):** Realistic materials (albedo, metallic, roughness). Standard in games/films by 2025.

5.4 Texturing

- **Texture Maps:** Diffuse (color), normal (lighting), specular, bump.
- **UV Mapping:** 2D texture to 3D model.
- **Mipmapping:** Precomputed texture scales to reduce aliasing.
- **Procedural Textures:** Generated algorithmically (e.g., Perlin noise).

6 Animation

6.1 Keyframe Animation

- Define poses at timestamps, interpolate (linear, spline).
- Tools: Blender, Unity.

6.2 Skeletal Animation

- Bones control mesh (skinning). Used in games (e.g., character rigging).

6.3 Physics-Based Animation

- Simulate dynamics (e.g., cloth, fluids). Libraries: PhysX, Houdini.

6.4 Motion Capture

- Record human movements for realistic animation (e.g., Vicon systems).

7 Graphics APIs and Tools

7.1 APIs

- **OpenGL:** Cross-platform, widely used.
- **DirectX (12):** Microsoft, game-focused.
- **Vulkan:** Low-level, high-performance (successor to OpenGL).
- **WebGL:** Web-based (e.g., Three.js).
- **Metal:** Apple's GPU framework.

7.2 Engines and Frameworks

- **Game Engines:** Unreal Engine 5, Unity (real-time, ray tracing).
- **Rendering:** Blender (offline), V-Ray, Arnold (films).
- **Libraries:** OpenCV (vision), SFML (2D), GLFW.

8 Advanced Topics

8.1 Real-Time Rendering

- **Techniques:** Deferred shading, shadow mapping, ambient occlusion.
- **Optimizations:** Level of Detail (LOD), frustum culling, occlusion culling.
- **Modern:** DLSS (Deep Learning Super Sampling), AMD FSR for upscaling.

8.2 Ray Tracing and Global Illumination

- **Real-Time Ray Tracing:** NVIDIA RTX, AMD RDNA 3 (2025 standard in games).
- **Path Tracing:** Monte Carlo for films (e.g., Pixar's RenderMan).

8.3 Virtual and Augmented Reality

- **VR:** Headsets (Meta Quest 3, 2025 models with eye tracking).
- **AR:** Overlay graphics (e.g., HoloLens, Apple Vision Pro).
- **Challenges:** Low latency, high resolution, motion sickness.

8.4 Neural Rendering

- **NeRF (Neural Radiance Fields):** 3D reconstruction from 2D images using neural nets.
- **AI-Driven:** Generative AI for textures, models (e.g., Stable Diffusion for assets).

8.5 Quantum Graphics

- **Emerging:** Quantum algorithms for rendering (e.g., Grover's for search in scene graphs).
- **Applications:** Optimization in ray tracing, material simulation.

8.6 Sustainable Graphics

- **Focus (2025):** Energy-efficient rendering for edge devices, low-power GPUs.

9 Graphics Programming

Example (OpenGL with GLSL for a triangle):

```
#include <GLFW/glfw3.h>
void render() {
    glClear(GL_COLOR_BUFFER_BIT);
    glBegin(GL_TRIANGLES);
    glVertex2f(0.0f, 0.5f); // Top
    glVertex2f(-0.5f, -0.5f); // Bottom-left
    glVertex2f(0.5f, -0.5f); // Bottom-right
    glEnd();
}
int main() {
    glfwInit();
    GLFWwindow* window = glfwCreateWindow(800, 600, "Triangle", NULL,
    NULL);
    glfwMakeContextCurrent(window);
    while (!glfwWindowShouldClose(window)) {
        render();
        glfwSwapBuffers(window);
        glfwPollEvents();
    }
    glfwTerminate();
    return 0;
}
```

10 Applications

- **Gaming:** AAA titles (Cyberpunk 2077), indie games (Unity).
- **Film:** CGI in Avatar, Marvel films.
- **Simulation:** Flight simulators, medical training.
- **Visualization:** Scientific (molecular), architectural (CAD).
- **VR/AR:** Metaverse, industrial training.
- **AI Integration:** Generative assets, real-time upscaling.

11 Best Practices

- **Optimization:** Minimize draw calls, use instancing.
- **Portability:** Write shaders for multiple APIs (GLSL, HLSL).
- **Testing:** Debug with tools like RenderDoc, NVIDIA Nsight.
- **Realism:** Use PBR, high-quality textures.
- **Energy Efficiency:** Optimize for mobile/edge devices.